

Qayd: Yet Another Constraint Solver with Lazy Clause Generation, Written in Rust

Sohaib Afifi

Univ. Artois, UR 3926, Laboratoire de Génie Informatique et
d'Automatique de l'Artois (LGI2A), F-62400 Béthune, France
sohaib.lafifi@univ-artois.fr

Version 0.8.0, June 2026

Abstract

Qayd is an open-source constraint-programming solver written in Rust for constraint satisfaction problems and mono-objective constraint optimization problems over finite-domain integer variables. The v0.8.0 release targets the XCSP³-core catalogue and combines event-driven finite-domain propagation with Lazy Clause Generation (LCG). Complete modes use conflict-driven search and branch-and-bound, while parallel and incumbent-oriented modes add portfolio search, selective information sharing, Large Neighborhood Search, and incremental local search. This note describes the solver artifact submitted to the 2026 XCSP³ Competition.

1 Frontend and propagation kernel

The XCSP³ frontend is built on a revision-pinned Rust parser and posts each accepted callback directly into the solver. It reads plain XML as well as `.lzma` and `.xz` files, handles CSP and mono-objective COP instances, and emits the competition status, objective, and instantiation records. The supported catalogue is a broad subset of XCSP³-core [1, 2]. It includes intensional and extensional constraints, **regular**, **mdd**, comparison and counting globals, extrema and **element**, channeling, packing, scheduling, **circuit**, **instantiation**, and parser-expanded **slide**. Unsupported shapes are rejected explicitly. In particular, product and lexicographic objectives are outside the v0.8.0 frontend.

Domains are selected from three trailed representations: a dense sparse-set for compact ranges, an explicit-support sparse-set for sparse domains, and reversible bounds with trailed interior holes for very wide ranges. Propagators subscribe at fix, bound-change, or domain-change granularity. A FIFO queue and duplicate suppression drive propagation to a fixpoint, and root propagation removes fixed non-objective variables before search.

Qayd provides dedicated filtering for several important globals. Positive tables use Compact-Table-style reversible bitsets and residues [3]. **regular** and **mdd** use layered forward-backward reachability, following the automaton view of [4]. **allDifferent** enforces Régim's matching-based domain consistency [5]. Fixed-resource **cumulative** combines time-tabling, overload checks, and edge-finding. Other accepted forms are implemented by dedicated propagators or sound decompositions.

2 Learning and complete search

The exact search layer follows LCG [6]. Integer-domain facts are exposed as equality atoms $[x = v]$ and order atoms $[x \geq k]$. Atoms for sparse domains are allocated only for supported values, while atoms for wide ranges are created on demand. Propagators attach explicit premises to domain changes when available; otherwise the engine constructs a sound explanation from the pre-propagation scope. Clausal propagation uses two watched literals and is interleaved with finite-domain propagation.

Conflicts are analyzed at the first unique implication point. Qayd learns an asserting clause, backjumps non-chronologically, and maintains variable activity and Literal Block Distance (LBD). Search combines dom/wdeg [7] with VSIDS activity by minimizing a domain-size to combined-score ratio. Phase saving, seed-dependent endpoint decisions, Luby restarts, an LBD-degradation restart trigger [8], learned-clause reduction, bounded root probing, and short CP-aware clause vivification complete the default configuration. A lone sequential search periodically inverts its default polarity for one restart segment; portfolio workers instead obtain diversity from their seeds and restart schedules.

A single-worker CSP run uses CDCL to find one solution or prove unsatisfiability. COP uses CDCL branch-and-bound and posts a strictly improving bound after each incumbent. The objective can remain a variable, a symbolic linear form, or an expression, which avoids materializing wide objective ranges solely for search.

3 Parallel and incumbent-oriented modes

Multiworker CSP solving is heterogeneous: one worker performs chronological depth-first search, while the remaining workers use diversified CDCL. CDCL workers exchange only learned clauses of length at most eight and LBD at most four. The first solution or complete unsatisfiability proof terminates the portfolio.

Parallel COP workers share the incumbent value atomically. When the objective is materialized as a variable, proof workers may also exchange short low-LBD clauses. Additional roles include disjoint proof-space splitting, optimistic objective probes, adaptive incumbent-driven Large Neighborhood Search [9], and a local-search incumbent generator. Heuristic incumbents used by the proof portfolio are checked by fixing them in a cloned finite-domain model and propagating before they update the shared bound.

The incumbent-only modes never report an optimality proof. With one worker, `--fast-cop` uses the exact engine with objective-biased value choices; with several workers it dispatches local search. `--turbo` always uses local search and enables Guided Local Search weights together with a bounded min-conflicts candidate set for large domains. The local-search engine maintains incremental violation scores, uses constructive starts for selected table and element patterns, repairs Boolean exact-cover rows preferentially, and applies randomized restarts.

4 Competition interface and availability

The common invocation is:

```
qayd -t SECONDS --seed SEED -p THREADS [mode] instance.xml[.lzma|.xz]
```

where `mode` can select `--fast-cop`, `--turbo`, or optional COP roles such as `--split`, `--probe`, and `--lns`. If explicit values are absent, Qayd reads `RANDOMSEED` and `NBCORE`. The release is identified by Git tag `v0.8.0` and commit `e72a9e10c8d8c01ce37b5ae2ec379f78f322ff42`. Source code and platform binaries are available at <https://github.com/sohaibafifi/qayd/releases/tag/v0.8.0> under the MIT License.

References

- [1] F. Boussemart, C. Lecoutre, G. Audemard, and C. Piette. XCSP3: An integrated format for benchmarking combinatorial constrained problems. *arXiv:1611.03398*, 2016.
- [2] F. Boussemart, C. Lecoutre, G. Audemard, and C. Piette. XCSP3-core: A format for representing constraint satisfaction and optimization problems. *arXiv:2009.00514*, 2020.
- [3] J. Demeulenaere, R. Hartert, C. Lecoutre, G. Perez, L. Perron, J.-C. Régin, and P. Schaus. Compact-Table: Efficiently filtering table constraints with reversible sparse bit-sets. In *CP 2016*, pages 207-223.
- [4] G. Pesant. A regular language membership constraint for finite sequences of variables. In *CP 2004*, pages 482-495.
- [5] J.-C. Régin. A filtering algorithm for constraints of difference in CSPs. In *AAAI 1994*, pages 362-367.
- [6] T. Feydy and P. J. Stuckey. Lazy clause generation reengineered. In *CP 2009*, pages 352-366.
- [7] F. Boussemart, F. Hemery, C. Lecoutre, and L. Saïs. Boosting systematic search by weighting constraints. In *ECAI 2004*, pages 146-150.
- [8] G. Audemard and L. Simon. Predicting learnt clauses quality in modern SAT solvers. In *IJCAI 2009*, pages 399-404.
- [9] P. Shaw. Using constraint programming and local search methods to solve vehicle routing problems. In *CP 1998*, pages 417-431.